

Research Project: Investigation of numerical schemes for the Porous Medium Equation

Nikolaus Vohwinkel

Abstract

The porous medium equation (PME) is a nonlinear partial differential equation that models diffusion in porous media. In this Research Project, the Local Discontinuous Galerkin (LDG) method is employed to numerically solve the PME. The well-established method is tested in various settings, using different polynomial orders of basis functions, to study their effects on the solution's behavior. Furthermore, a hyperbolic system that degenerates into the PME under certain parameter regimes is investigated. For both approaches, extensive numerical results are presented to illustrate the capabilities and limitations of each method.

1 Introduction

The porous medium equation (PME) is a nonlinear diffusion equation that models various physical and biological phenomena, such as isentropic gas flow through porous media, nonlinear heat conduction, and even image processing. The unknown function $u(x, t)$ typically represents a nonnegative quantity such as concentration, density, or temperature. The general form of the PME is:

$$\frac{\partial u}{\partial t} = \Delta(u^m), \quad x \in \Omega, \quad t > 0,$$

where $m > 1$ is a constant characterizing the medium's porosity. Assuming that the initial condition u_0 is nonnegative, the PME can be rewritten as

$$\frac{\partial u}{\partial t} = \Delta(u^m) = \nabla \cdot (mu^{m-1} \nabla u) = \nabla \cdot (a(u) \nabla u), \quad x \in \Omega, \quad t > 0,$$

where $a(u) = mu^{m-1} \geq 0$. Thus, the PME can be identified as a nonlinear diffusion equation of the degenerate parabolic type, since it degenerates when $u = 0$. Notably, if the initial data has compact support, the solution of the initial value problem for the PME will also have compact support at any given time $t > 0$. Due to this feature, the PME exhibits finite speed of propagation at the moving front or “free boundary” region where the solution vanishes.

Another interesting novelty of the PME is the so-called waiting time phenomenon, which can occur for specially chosen initial data. In this case, the free boundary remains stationary until a finite positive time (the waiting time), after which it instantaneously moves with finite speed, e.g. see chapter 4.

A fundamental example of a solution is the Barenblatt (or source-type) solution [1], which can be written explicitly as a formula which is given by

$$B_m(x, t) = t^{-k} \left[\left(1 - \frac{k(m-1)}{2m} \frac{|x|^2}{t^{2k}} \right)_+ \right]^{1/(m-1)}, \quad (1)$$

where $(u)_+ = \max(u, 0)$ and $k = 1/(m+1)$. For any time $t > 0$, this solution has compact support $[-\alpha_m(t), \alpha_m(t)]$, with the interface/free boundary moving outward at finite speed $\alpha_m(t)$, where

$$\alpha_m(t) = \sqrt{\frac{2m}{k(m-1)}} \cdot t^k.$$

This solution will later be useful for assessing the accuracy of the numerical methods presented. In this work, two numerical frameworks will be tested. One relies on the Local Discontinuous Galerkin (LDG) formulation proposed by Zhang and Wu [9], while the other employs a hyperbolic system that degenerates into the PME as the parameter $\varepsilon \rightarrow 0$, similar to the system introduced by Jin, Pareschi, and Toscani [4]. Both approaches utilize a Galerkin ansatz on discontinuous basis functions of varying polynomial degree. Since this ansatz tends to produce oscillations near the free boundary, the LDG method introduces an artificial diffusion parameter γ and employs limiters extensively in the vicinity of the free boundary to address this issue. However, for the hyperbolic system, these oscillations are unavoidable, as will be demonstrated by the findings in this work.

2 Mathematical and Numerical Formulation

To numerically solve the porous medium equation (PME), we discretize the spatial domain using a finite number of non-overlapping cells and apply a high-order accurate numerical method tailored for problems with nonlinear diffusion and potential degeneracies. Specifically, we employ the Discontinuous Galerkin (DG) method, known for its flexibility in handling complex boundary conditions, its ability to use high-order polynomial approximations, and its local conservation properties.

The computational domain is $\Omega \subset \mathbb{R}$ divided into $N_{\text{cells}} \in \mathbb{N}$ cells, where each cell

$$I_i = \left(x_{i-\frac{1}{2}}, x_{i+\frac{1}{2}}\right)$$

has width

$$h_i = x_{i+\frac{1}{2}} - x_{i-\frac{1}{2}}, \quad i = 1, \dots, N_{\text{cells}}.$$

For notation purposes $x_{i\pm\frac{1}{2}}$ denotes the position at the interfaces and x_i denotes the mid point of the cell i .

Since we use the DG approach, we adopt an ansatz that separates spatial and temporal dependencies of the solution. Within each cell I_i , the approximate solutions are represented as

$$u_h(x, t)|_{I_i} = \sum_{l=0}^n u_i^l(t) \varphi_h^l(x), \quad j_h(x, t)|_{I_i} = \sum_{l=0}^n j_i^l(t) \varphi_h^l(x), \quad q_h(x, t)|_{I_i} = \sum_{l=0}^n q_i^l(t) \varphi_h^l(x),$$

with j, q the auxiliary variables (see chapter 2.1 and 2.2). The basis functions φ_h^l are scaled Legendre polynomials defined by

$$\varphi_h^l(x) = p_l \left(\frac{2(x - x_i)}{h_i} \right), \quad x \in I_i,$$

and p_l denotes the Legendre polynomial of degree $l \in \mathbb{N}_0$ and $\frac{2(x-x_i)}{h_i} \in [-1, 1]$ is the scaled argument in terms of the current cell width. For the initial solution $u_h(0)$, we use

the L^2 -projection of the given initial data u_0 into the discrete function space

$$V_h^n = \{v_h : \mathbb{R} \rightarrow \mathbb{R} \mid v_h|_{I_i} \in \mathcal{P}_n(I_i), \forall i = 1, \dots, N_{\text{cells}}\},$$

where $\mathcal{P}_n(I_i)$ denotes the space of polynomials of degree at most n on the cell I_i . The initial projection satisfies

$$\int_{I_i} (u_h(0, x) - u_0(x)) \varphi_h(x) dx = 0, \quad \forall \varphi_h \in V_h^n, \quad \forall i = 1, \dots, N_{\text{cells}}.$$

The Legendre polynomials and their derivatives are computed using

Algorithm 22: **LegendrePolynomialAndDerivative** from Kopriva [5]. To accurately evaluate integrals, we employ Gauss–Legendre quadrature with nodes and weights obtained from Algorithm 23: **LegendreGaussNodesAndWeights** (also from Kopriva), allowing for flexible polynomial degrees.

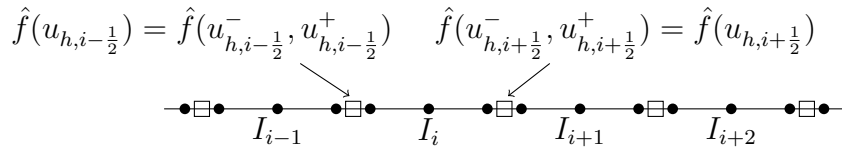


Fig. 1: Example cells with Gauss nodes and interface fluxes for polynomial degree $n = 3$.

2.1 The Local Discontinuous Galerkin (LDG) Method

A common strategy for addressing the nonlinearity in the porous medium equation (PME) is to introduce auxiliary variables, which facilitate the construction of a Local Discontinuous Galerkin (LDG) scheme, as proposed by Cockburn and Shu [2]. By carefully selecting appropriate numerical fluxes at element interfaces, the resulting formulation has been proven to be L^2 -stable (see Chapter 2.3 in [2]).

To apply the LDG method, we first rewrite the PME in first-order form by introducing auxiliary variables g and q :

$$\begin{aligned} \partial_t u + \partial_x \left[f(u) - \sqrt{a(u)} q \right] &= s(u), & x \in I, \quad t > 0, \\ q - \partial_x g &= 0, & x \in I, \quad t > 0, \end{aligned}$$

where

$$g(u) = \int_0^u \sqrt{a(s)} ds, \quad \text{with} \quad a(u) = m u^{m-1}.$$

Following the proposed LDG approach by Cockburn and Shu, Zhang and Wu [9] introduced the following numerical fluxes to handle discontinuities at element interfaces. In conjunction with the spatial DG discretization, the semi-discrete formulation in a cell I_i reads as:

$$\begin{aligned} \partial_t u_i^l(t) &= \frac{2l+1}{h} \int_{I_i} \left[s(u_h) \varphi^l(x) + \left(f(u_h) - \sqrt{a(u_h)} q_h \right) \partial_x \varphi^l(x) \right] dx \\ &\quad - \frac{2l+1}{h} \left[\left(\hat{f}(u_h) - \frac{\llbracket g(u_h) \rrbracket}{\llbracket u_h \rrbracket} \llbracket q_h \rrbracket - \gamma \llbracket q_h \rrbracket \right) \varphi^l(x) \right]_{x_{i-\frac{1}{2}}}^{x_{i+\frac{1}{2}}}, \\ q_i^l(t) &= -\frac{2l+1}{h} \int_{I_i} g(u_h) \partial_x \varphi^l(x) dx + \left[(-\llbracket g(u_h) \rrbracket + \gamma \llbracket u_h \rrbracket) \varphi^l(x) \right]_{x_{i-\frac{1}{2}}}^{x_{i+\frac{1}{2}}}, \end{aligned} \quad (2)$$

where the arithmetic mean and jump operators are defined at cell interfaces as:

$$\{g(u_h)\} = \frac{g(u_h^+) + g(u_h^-)}{2}, \quad \llbracket u_h \rrbracket = u_h^+ - u_h^-.$$

When $\llbracket u_h \rrbracket \neq 0$, the term

$$\frac{\llbracket g(u_h) \rrbracket}{\llbracket u_h \rrbracket} \quad (3)$$

is well-defined and serves as an approximation of $\sqrt{a(u_h)}$. However, when $\llbracket u_h \rrbracket = 0$, this expression becomes undefined. To preserve numerical stability and ensure a meaningful evaluation in such cases, it is customary to replace the undefined ratio with

$$\sqrt{a(u_h)}.$$

The numerical behavior of this replacement is highly sensitive to how equality is determined between u_h^+ and u_h^- . To robustly assess whether these two values are approximately equal within machine precision, we adopt the adaptive tolerance criterion proposed by Kopriva [5], specifically Algorithm 139: **AlmostEqual**. This method enhances the stability and reliability of the scheme by applying a practical and adaptive threshold for floating-point comparisons.

2.2 Hyperbolic Balance law System

Another approach to numerically solve the porous medium equation is to consider a hyperbolic system that degenerates to the porous medium equation as $\varepsilon \rightarrow 0$, similar to the method proposed by Jin, Pareschi, and Toscani in [4]. To this end, we introduce an auxiliary variable $\mathbf{j}$ and a relaxation parameter ε , leading to the following system of hyperbolic balance laws:

$$\begin{aligned} \partial_t u^\varepsilon + \nabla \cdot \mathbf{j}^\varepsilon &= 0, \\ \varepsilon \partial_t \mathbf{j}^\varepsilon + \nabla \cdot ((u^\varepsilon)^m) &= -\mathbf{j}^\varepsilon. \end{aligned} \quad (4)$$

As already mentioned, we use for the spatial discretization the Discontinuous Galerkin (DG) method incorporation with scaled Legendre polynomials as basis functions. This yields the following semi-discrete formulation given for cell I_i

$$\begin{aligned} \int_{I_i} \partial_t u_h(x, t) \varphi^r(x) dx + \int_{I_i} \partial_x j_h(x, t) \varphi^r(x) dx &= 0, \\ \int_{I_i} \varepsilon \partial_t j_h(x, t) \varphi^r(x) dx + \int_{I_i} \partial_x (u_h^m) \varphi^r(x) dx &= - \int_{I_i} j_h \varphi^r(x) dx. \end{aligned}$$

Projecting onto the Legendre basis and exploiting orthogonality, we obtain the following system of ODEs for the modal coefficients $u_i^l(t)$ and $j_i^l(t)$:

$$\begin{aligned} \int_{I_i} \partial_t \left(\sum_{l=0}^n u_i^l(t) \varphi^l(x) \right) \varphi^r(x) dx + \int_{I_i} \partial_x \left(\sum_{l=0}^n j_i^l(t) \varphi^l(x) \right) \varphi^r(x) dx &= 0, \\ \int_{I_i} \varepsilon \partial_t \left(\sum_{l=0}^n j_i^l(t) \varphi^l(x) \right) \varphi^r(x) dx + \int_{I_i} \partial_x (u_h^m) \varphi^r(x) dx &= - \int_{I_i} j_h \varphi^r(x) dx. \end{aligned}$$

$\Leftrightarrow$

$$\begin{aligned}\partial_t u_i^l(t) &= \frac{2l+1}{h_i} \int_{I_i} j_h \partial_x \varphi_h^l dx - \frac{2l+1}{h_i} \left[\widehat{j}_h \varphi_h^l \right]_{x_{i-\frac{1}{2}}}^{x_{i+\frac{1}{2}}}, \\ \varepsilon \partial_t j_i^l(t) &= -\frac{2l+1}{h_i} \int_{I_i} j_h \varphi_h^l dx + \frac{2l+1}{h_i} \int_{I_i} (u_h)^m \partial_x \varphi_h^l dx - \frac{2l+1}{h_i} \left[\widehat{u}_h^m \varphi_h^l \right]_{x_{i-\frac{1}{2}}}^{x_{i+\frac{1}{2}}},\end{aligned}\tag{5}$$

where $\widehat{j}_h$ and $\widehat{u}_h$ are appropriate numerical fluxes defined at the cell interfaces to ensure stability and consistency. This formulation is then solved in time with the help of an explicit Runge-Kutta time stepping scheme (chapter 2.4).

2.3 Limiter

In order to obtain accurate and stable simulations, two limiters are introduced, both of which utilize a first-order L^2 -projection of u_h . To implement this projection, we employ a least-squares approach that remains as close as possible to the original numerical solution, thereby ensuring that the projection remains approximately mass-conserving.

2.3.1 First-Order Slope Limiter

The general formulation of the first-order slope-limited approximation is given by

$$u_h|_{I_i} = \bar{u}_i + (x - x_i)\sigma_{x,i}, \quad i = 1, 2, \dots, N_{\text{cells}},$$

where $\sigma_{x,i}$ is the slope, and $\bar{u}_i$ is the average solution in cell I_i . Using the minmod slope limiter proposed by Osher [7] with a slight modification where we accept values under a certain threshold:

$$\Lambda \Pi_h^1 u_h|_{I_i} = \bar{u}_i + (x - x_i)\bar{m} \left(u_{x,i}, \frac{\bar{u}_i - \bar{u}_{i-1}}{h_i/2}, \frac{\bar{u}_{i+1} - \bar{u}_i}{h_i/2} \right),$$

where

$$\bar{m}(a_1, a_2, a_3) = \begin{cases} a_1, & \text{if } |a_1| \leq \mu h^2, \\ s \min(|a_1|, |a_2|, |a_3|), & \text{if } |a_1| > \mu h^2 \text{ and } s = \text{sgn}(a_k), \quad k = 1, 2, 3, \\ 0, & \text{otherwise,} \end{cases}$$

here $\mu \geq 0$ has to be chosen in such a way that the total variation boundedness (TVB) and high order accuracy is ensured, for this work only $\mu = 1$ was used. The limiter is applied in such a way that it preserves the current boundary values of the solution unless their deviation from the cell average is too large. This enforcement is given by:

$$\begin{aligned}\Lambda \Pi_h^1 u_{h,i+1/2}^- &= \bar{u}_i + \bar{m}(u_{i+1/2}^- - \bar{u}_i, \bar{u}_i - \bar{u}_{i-1}, \bar{u}_{i+1} - \bar{u}_i), \\ \Lambda \Pi_h^1 u_{h,i-1/2}^+ &= \bar{u}_i - \bar{m}(\bar{u}_i - u_{i-1/2}^+, \bar{u}_i - \bar{u}_{i-1}, \bar{u}_{i+1} - \bar{u}_i).\end{aligned}$$

If the following conditions hold:

$$\Lambda \Pi_h^1 u_{h,i+1/2}^- = u_{i+1/2}^- \quad \text{and} \quad \Lambda \Pi_h^1 u_{h,i-1/2}^+ = u_{i-1/2}^+,$$

then no limiting is needed. Otherwise, a local first-order L^2 -projection u_h^1 of u_h is computed, and the limiter $\Lambda \Pi^1$ is reapplied.

2.3.2 Second Limiter: Positivity Preservation

The slope limiter alone does not guarantee non-negativity within the cell, which is essential due to the presence of square roots in the model. To enforce positivity, a second limiting step is introduced:

1. If any value in a cell is negative, compute a local first-order L^2 -projection u_h^1 of u_h .
2. Calculate the average value $\bar{u}_i$ and apply the second limiter $P\Pi_h^1 u_h^1$.

This second limiter reconstructs a linear function based on the cell average to ensure positivity throughout the cell:

$$P\Pi_h^1 u_h^1 = \begin{cases} \left[1 - \frac{2(x-x_i)}{h_i}\right] \bar{u}_i, & \text{if } u_{h,i+1/2}^- < 0, \\ \left[1 + \frac{2(x-x_i)}{h_i}\right] \bar{u}_i, & \text{if } u_{h,i-1/2}^+ < 0. \end{cases}$$

This reconstruction ensures that the solution is bounded, continuous, and positive throughout the cell. The general L^2 -projected linear function takes the form:

$$f(x) = \left(u_{i+1/2}^- - u_{i-1/2}^+\right) \frac{x - x_i}{h_i} + \frac{1}{2} \left(u_{i+1/2}^- + u_{i-1/2}^+\right).$$

Assuming only one interface value is negative, we modify the solution such that:

$$\bar{u}_i = \frac{1}{2} \left(u_{i+1/2}^- + u_{i-1/2}^+\right),$$

which preserves the cell average and ensures:

$$2|\bar{u}_i| < u_{i-1/2}^+ \quad \text{or} \quad 2|\bar{u}_i| < u_{i+1/2}^-.$$

This ensures that the reconstruction does not introduce new extrema. Since the negative interface value is clamped to zero, the total variation does not increase, thus maintaining the TVD (total variation diminishing) property.

Handling Negative Cell Averages While Zhang and Wu [9] showed that their method avoids negative cell averages for polynomial degree $l = 0$, the numerical experiments provided in this work indicate this does not always hold in practice for varying polynomial degrees. Therefore, as a final safeguard:

$$P\Pi_h^1 u_h^1 = \begin{cases} \left[1 - \frac{2(x-x_i)}{h_i}\right] \bar{u}_i, & \text{if } u_{h,i+1/2}^- < 0, \\ \left[1 + \frac{2(x-x_i)}{h_i}\right] \bar{u}_i, & \text{if } u_{h,i-1/2}^+ < 0, \\ 0, & \text{if } \bar{u}_i < 0, \end{cases}$$

is used to avoid such cases.

2.4 Time-stepping

For time discretization of the semi-discrete Discontinuous Galerkin (DG) formulation, a third-order explicit Runge-Kutta scheme is utilized. The spatial DG scheme (Eq.:2 and Eq.:5) can be written as a first-order ODE system

$$\frac{du_h}{dt} = \mathbb{M}^{-1} \text{RHS}(u_h) = \mathcal{L}_h(u_h), \quad t \in (0, T),$$

where $\mathbb{M}$ is the mass matrix and $\text{RHS}(u_h)$ represents the spatial DG discretization operator. The third-order Runge-Kutta update from time t^n to $t^{n+1} = t^n + \Delta t^n$ is then given by

$$\begin{aligned} u^{(1)} &= u_h^n + \Delta t^n \mathcal{L}_h(u_h^n), \\ u^{(2)} &= \frac{3}{4}u_h^n + \frac{1}{4}u^{(1)} + \frac{1}{4}\Delta t^n \mathcal{L}_h(u^{(1)}), \\ u_h^{n+1} &= \frac{1}{3}u_h^n + \frac{2}{3}u^{(2)} + \frac{2}{3}\Delta t^n \mathcal{L}_h(u^{(2)}). \end{aligned}$$

The time step Δt^n must satisfy appropriate stability constraints, which differ depending on the numerical scheme. These conditions are expressed as:

$$\text{LDG: } \frac{\max |f'(u_h^n)| \Delta t^n}{\Delta x} + \frac{2 \max |a(u_h^n)| \Delta t^n}{(\Delta x)^2} \leq CFL \cdot \frac{1}{l+1}, \quad (6)$$

$$\text{Hyperbolic system: } \frac{\max |u_h^n| \Delta t^n}{\Delta x \sqrt{\varepsilon}} + \frac{2 \max |j_h^n| \Delta t^n}{(\Delta x)^2 \varepsilon} \leq CFL \cdot \frac{1}{l+1}, \quad (7)$$

where l denotes the maximal polynomial degree of the basis functions, and CFL is the Courant–Friedrichs–Lewy number selected according to stability requirements.

In line with Zhang and Wu [9], a CFL number of 0.02 was adopted for the LDG method with polynomial degree $l = 2$, and this value was retained throughout this work even for higher-order polynomials. For the hyperbolic system, a substantially larger value of $CFL = 0.9$ was found to yield stable results in all tested scenarios.

This choice stands in contrast to the more restrictive CFL numbers reported by Sigal and Shu [3], particularly those presented in their Table 2.1. However, the increased CFL value is justified in the current study, since the time step scales with $\Delta t/(\Delta x)^2$, and stability is maintained provided that the grid spacing satisfies $\max \Delta x_i < 1$ for all $1 \leq i \leq N_{\text{cells}}$.

3 Practical Implementation

For the implementation, C++ version: (Ubuntu 13.3.0-6ubuntu2 24.04) 13.3.0 was used along with the standard `vector` and `cmath` libraries for numerical operations, and the VTK library to generate output data. All calculations were performed in double precision. Additionally, for polynomial degrees $l > 15$, a more accurate algorithm for computing Gauss–Legendre nodes must be implemented.

```

Output Name: BBM2N160PP5
Polynomial Degree:  $l = 5$ 
Physical Domain:
Start: -5
End: 5
End Time: 1.0
Numerical Parameters:
Flux Function: barenblatt
Riemann Solver: upwind
Epsilon: 0.0
Limiter Parameters:
Use LimiterType: minmod Options: minmod, superbee, vanLeer, vanAlbada,
none
Use Limiter: true
Use Modified Limiter: true
Time Stepping:
CFL: 0.02
Scheme: RK (Runge-Kutta)
Maximum  $\Delta t$ :  $1 \times 10^{-6}$ 
Output:
Write State Frequency: 10000
Initial Condition:
Type: barenblatt
Options: gaussian, unitStep, negativeUnitStep, barenblatt, divorce,
sinus, diffDoubleUnitStep, unitDoubleStep, waitingTime
Barenblatt Parameters:
 $M = 2$ 
Time: 1.0
Use Source: true
Use Transport: false
Transport Flux: linear
Error Parameters:
Error interval A: -1.5
Error interval B: 1.5
Discretization:
Number of Cells: 160

```

Fig. 2: Example parameter.ini file.

Furthermore, the code was tested using `long double` floating-point precision. However, this led to significant stability issues within the LDG formulation, maybe cause to the comparison process with the `Algorithm:AlmostEqual` for the evaluation of the jump-term Eq.3 . As a result, the use of `long double` was abandoned, and computations were reverted to standard `double` precision. Switching from the LDG formulation to the hyperbolic system is straightforward: it only requires changing the `FluxFunction` and setting the parameter $\varepsilon \neq 0$ and the `FluxFunction` parameter to everything else but `barenblatt`. This flexibility enables the simulation of a wide range of test cases and modifications to the porous medium equation. For example, one can include a source term of the form cu^p , and/or add a transport term, which can be specified via the `TransportFlux` option.

For all error calculations, a fixed evaluation grid consisting of 1000 points is used. This grid remains constant across all tests. The solution is evaluated at these points using the Legendre basis functions and the computed coefficients. The user can specify the interval over which the evaluation points are distributed (default: $[-1.5, 1.5]$).

Algorithm 1 Rough representation of the simulation loop

```

1: Input:  $N_{cells}$ ,  $m$ ,  $l$ ,  $u_0$ ,  $fluxFunction$ ,  $useSource$ ,  $useTransport$ ,  $numFlux$ ,  $CFL$ ,  $endTime$ ,  $LimiterType$ ,  $ErrorInterval$ ,  $\varepsilon$ ,
2: Initialize:
3:    $u \leftarrow 0$  {Solution array}
4:    $dx \leftarrow \frac{1}{N_x-1}$  {Grid spacing}
5:    $LegendreGaussNodesAndWeights(l)$  {calc. nodes&weights for the pol. deg.  $l$ }
6:    $InitialSolution(u_0)$  { $L^2$  projection of  $u_0$ }
7:    $callLimiter(u)$  {checks if a cell needs to be limited}
8:    $CalcDt()$  {calc.  $\Delta t$ }
9: Run Simulation:
10: while if time  $\leq$  endTime do
11:    $RungeKutta()$  {before each RK-stage the semi-discrete DG formulation is
                    calculated and after each RK-stage the Limiters are called}
12:    $CalcDt()$  {update  $\Delta t$ }
13:   if iteration % nWriteState == 0 then
14:      $writeSolution(u, time)$ 
15:      $calcError(u, time)$ 
16:   end if
17: end while
18:    $writeSolution(u, time)$ 
19:    $calcError(u, time)$ 

```

For visualization purposes, **ParaView** was used due to its efficient support for data interpretation, flexible visual customization, and its capability to generate video outputs directly from VTK data files.

The source code can be found on github with this link:

<https://github.com/Nikolaus1997/Forschungsmodul>

or with this QR-code:



4 Numerical Results

All numerical simulations were performed on an Intel i7-3600K processor equipped with 16 GB of DDR3 RAM. The computational domain was $[-6, 6]$, and unless stated otherwise, all error calculations were conducted on the subdomain $[-1.5, 1.5]$. For both numerical approaches (LDG/hyperbolic system) periodic boundary conditions were imposed.

Local Discontinuous Galerkin To initiate the simulation, one must select the artificial diffusion parameter γ . Zhang and Shu [9] proposed the following formulation:

$$\gamma_{j+\frac{1}{2}} = \begin{cases} \frac{1}{2} \frac{g_{j+1} - g_j}{u_{j+1} - u_j}, & \text{if } u_{j+1} \neq u_j, \\ \frac{1}{2} \sqrt{a(u_{h,j})}, & \text{otherwise.} \end{cases}$$

This approach has been analytically justified and is suggested to extend naturally to higher-order polynomial basis functions beyond the case of polynomial degree $l = 0$. In the following numerical experiments, no stability issues were encountered, even for high exponents such as $m = 8$ coupled with a high order polynomial like $l = 5$.

Case 1 For the first numerical experiment the Barenblatt solution (Eq.:1) was used as the initial condition at $t = 1$ s. As expected the numerical experiments show that the accuracy of the numerical scheme appears to heavily depend on the nonlinearity. Only for the odd exponent $m = 3$, the number of limiter calls are in comparison rather low, and the experimental order of convergence (EOC) was maintained. In contrast, for all even exponents (e.g $m = 2$), the number of limiter calls increases substantially compare Tab.:3 and Tab.:2, and no improvement in accuracy is observed with h -refinement. This

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	2.28e-04	-	6.45e-04	-	509	365	82
20	5.37e-05	2.09	9.30e-05	2.79	3375	404	328
40	5.23e-05	0.04	6.80e-05	0.45	8426	7851	1309
80	5.20e-05	0.01	5.75e-05	0.24	53905	23397	5234
160	5.19e-05	0.00	5.37e-05	0.10	233769	67510	20935
320	5.19e-05	0.00	5.29e-05	0.02	820520	393031	83738

Tab. 1: Error calculation results for $l = 2$, $m = 8$ for the Barenblatt function at $t = 1$ s as initial data and the end time $t = 1.05$ s

behavior is particularly noticeable under h -refinement, whereas for l/p -refinement, an improvement in accuracy is indeed achieved. The first error calculations (Tab. 1 and Tab. 2) in this work were performed up to the final time $t = 1.05$ s to enable direct comparison with the results of Zhang and Shu [9], who also used this final time for their error analysis. However, it should be noted that their approach disregarded the entire computational domain and instead imposed Dirichlet boundary conditions, computing errors only within the arch region and excluding areas containing kinks. Specifically, they focused solely on the domain $[-1.5, 1.5]$ with Dirichlet boundaries. In contrast, the present study computed the approximate solution over the entire domain $[-6, 6]$ and performed error calculations

exclusively within the subdomain $[-1.5, 1.5]$. This fundamental difference in the treatment of boundary conditions and domain extents leads to substantial discrepancies in the experimental order of convergence (EOC). Consequently, their assertion that “[...] the accuracy is preserved in the smooth part of the solution [...]” (Zhang and Shu [9], p. 140) cannot be reproduced when the entire domain $[-6, 6]$ is considered and an even exponent m is chosen. However, for odd exponents m , this statement does hold (see Tab. 2). Interestingly, the accuracy appears to be limited at an order of magnitude of 10^{-11} , beyond which no further improvement is observed (see Appendix 7). This behavior of the numerical scheme could be linked to the use of double-precision arithmetic coupled with the adaptive threshold for floating-point comparisons in this work.

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	9.15e-04	-	5.64e-03	-	235	95	32
20	4.32e-05	4.40	2.56e-04	4.46	1129	740	124
40	5.18e-06	3.06	4.37e-05	2.55	4947	2961	494
80	6.41e-07	3.01	4.98e-06	3.13	13940	11853	1976
160	8.07e-08	2.99	5.96e-07	3.06	75289	47413	7903
320	1.01e-08	3.00	7.29e-08	3.03	461077	189655	31610

Tab. 2: Error calculation results for the parameters $l = 2$, $m = 3$ and the final time $t = 1.05$ s with the Barenblatt function at $t = 1$ s as initial data.

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	6.65e-03	-	1.07e-02	-	3755	153	367
20	1.40e-03	2.25	1.60e-03	2.73	15916	7643	1467
40	1.08e-03	0.37	1.33e-03	0.27	62790	35200	5868
80	1.21e-03	-0.16	1.40e-03	-0.08	253359	140815	23472
160	1.23e-03	-0.03	1.41e-03	-0.01	1023085	563277	93884
320	1.25e-03	-0.02	1.42e-03	-0.01	4099218	2253158	375535

Tab. 3: Error calculation results for the parameters $l = 2$, $m = 2$ and the final time $t = 1.05$ s with the Barenblatt function at $t = 1$ s as initial data.

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.69e-01	-	6.67e-01	-	6081	561	514
20	2.25e-01	-0.41	1.25e+00	-0.91	31713	12386	2065
40	2.33e-01	-0.05	1.77e+00	-0.50	117348	49671	8279
80	2.57e-01	-0.14	3.11e+00	-0.82	498766	198603	33131
160	3.84e-01	-0.58	6.51e+00	-1.06	2137409	793967	132509
320	5.27e-01	-0.46	1.16e+01	-0.83	7894659	2829346	530013

Tab. 4: Error calculation results for the parameters $l = 2$, $m = 3$ and the final time $t = 2$ s and the Barenblatt function at $t = 1$ s as initial data

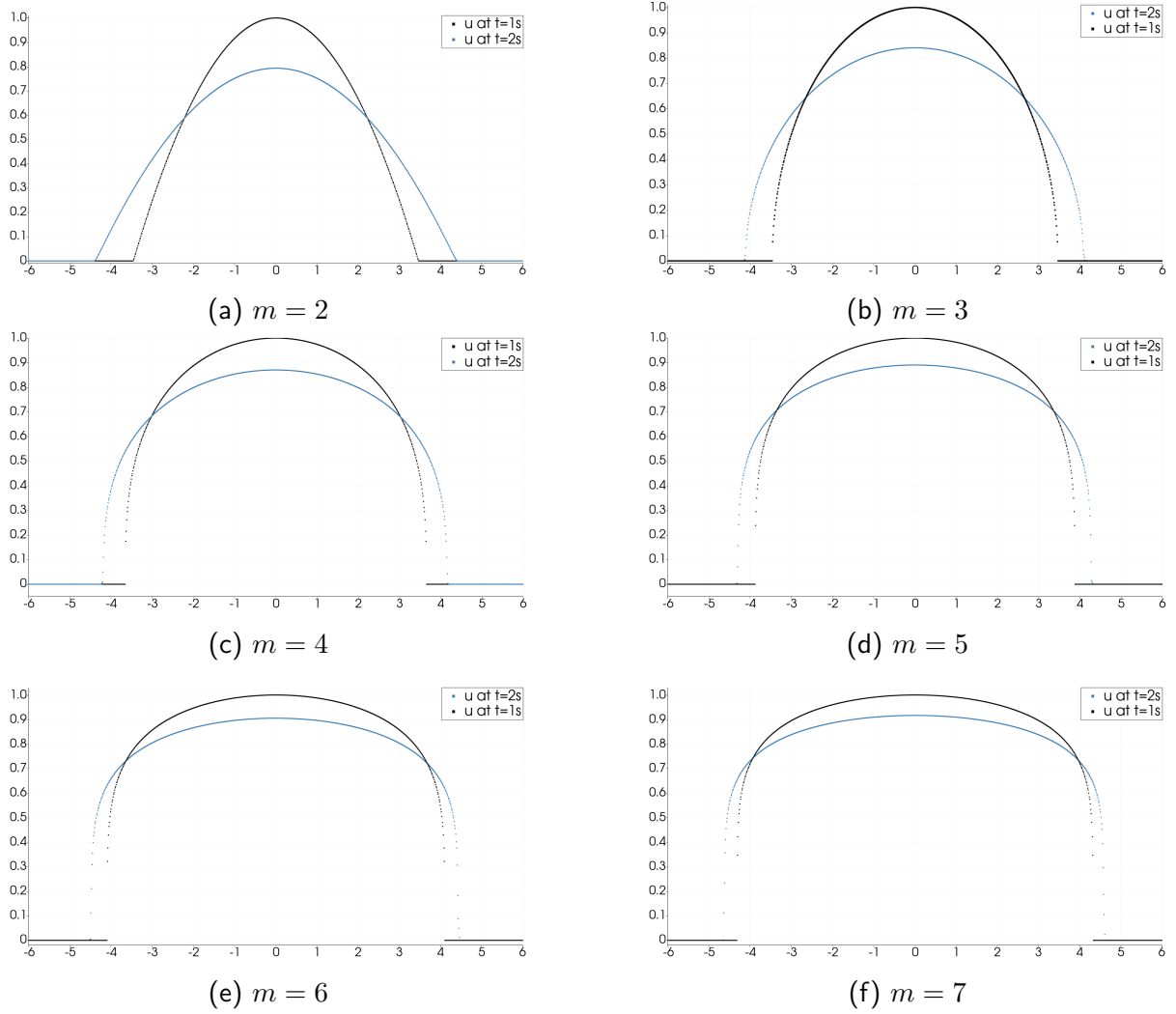


Fig. 3: Numerical results for the Barenblatt function as initial condition at $t = 1$ s computed until the final time $t = 2$ s. With the polynomial degree $l = 3$ and number of cells $N_{cells} = 320$ for various nonlinearities m .

The investigation of the solution's overall behavior with increasing nonlinearity up to time $t = 2$ s demonstrates consistency with the general form of the Barenblatt-type solution, as shown in Fig. 3. Additionally, the LDG formulation combined with the proposed limiters captures the free boundary with high accuracy. Nonetheless, both the L^2 and L^∞ errors are significantly larger, as expected, due to the simulation running over twenty times longer than the previous case and involving substantially more limiter activations. In this case not even l/p -refinement seem to increase the accuracy (in appendix Tab. 7).

Case 2 Since we have thus far considered only the porous medium equation without a source term, we now turn our attention to the case in which a strong absorption source term is added

$$\frac{\partial u}{\partial t} = (u^m)_{xx} - cu^p, \quad (8)$$

where $0 < p < 1$ and $0 < c$. Setting the initial condition to

$$u_0(x) = \begin{cases} |\sin x|, & x \in (-\pi, -\pi/6) \cup (\pi/6, \pi), \\ 0.5, & x \in (-\pi/6, \pi/6), \\ 0, & \text{otherwise,} \end{cases} \quad (9)$$

leads to a disconnect in the positive support region and after some time to a total absorption of the initial data see Fig. 4. The so-called "divorce of support" phenomenon, first described by Rosenau and Kamin [8] and later rigorously proven by Nakai and Tomoeda [6], can be reliably reproduced under the necessary parameter constraint $p+m=2$ with $0 < p < 1$ (in appendix Fig.10).

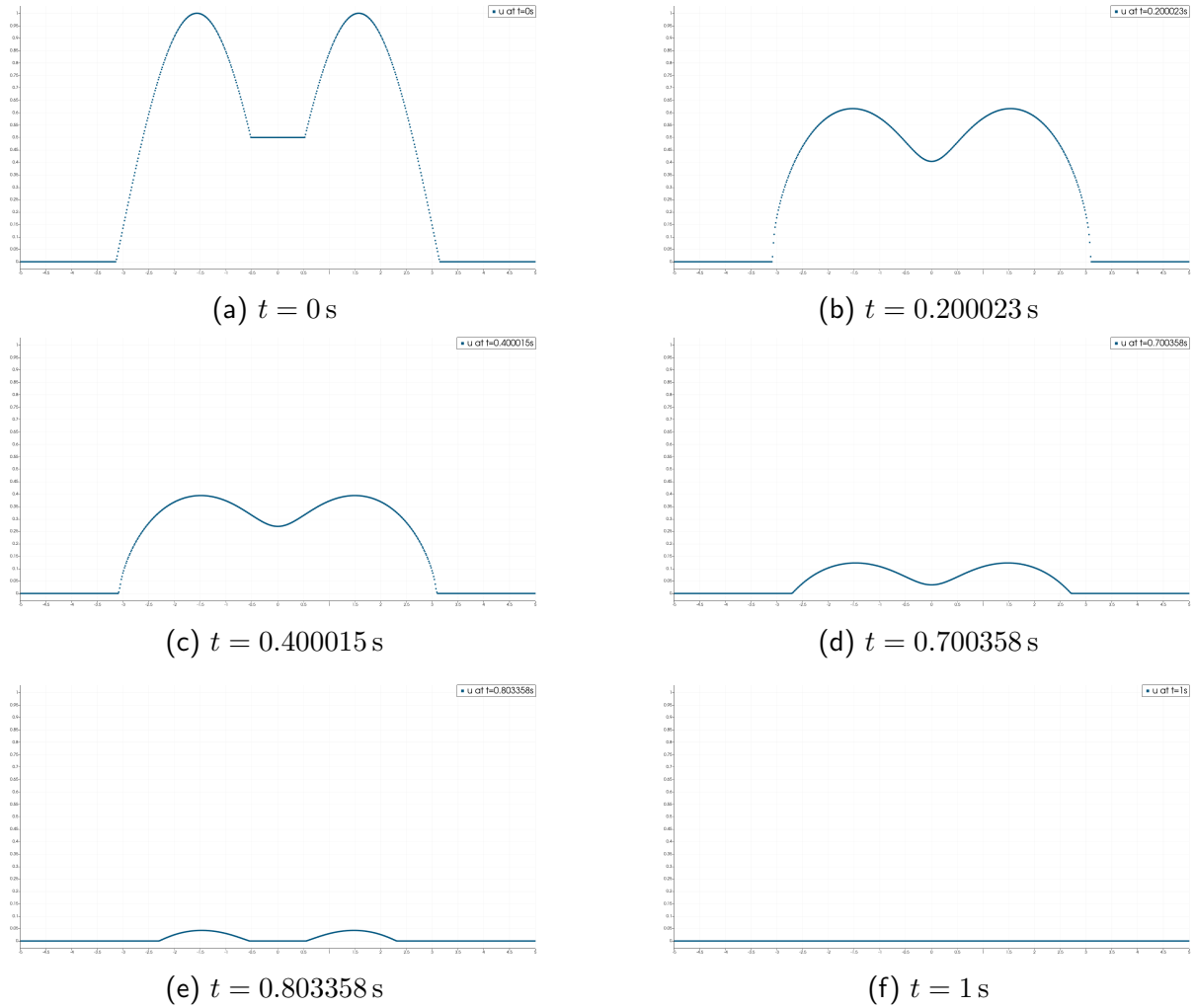


Fig. 4: Numerical results for the porous medium equation with strong absorption term with exponent $p = 0.01$, $m = 3$, $c = 1$ and polynomial degree $l = 3$

Hyperbolic System All numerical experiments for the hyperbolic system were conducted on the spatial domain $[-6, 6]$ using a uniform mesh with $N_{\text{cells}} = 320$, unless stated otherwise. The Courant–Friedrichs–Lewy (CFL) number was set to $CFL = 0.9$, and a central numerical flux was used to compute interface values at cell boundaries. The artificial parameter $\varepsilon = 0.01$ was selected to investigate the behavior of the porous medium equation, unless otherwise specified. Only for the experiment of varying ε the polynomial degree of $l = 2$ was chosen for all other tests the choice was $l = 3$.

Case 1 We consider the initial data

$$u_0(x) = \frac{\pi}{\sqrt{2}} \exp\left(-\frac{x^2}{2}\right),$$

which is smooth and compactly supported on the computational domain. The goal of this experiment is to examine whether the solution of the hyperbolic system converges to that of the porous medium equation as $\varepsilon \rightarrow 0$.

Fig. 5 illustrates the behavior of the solution for $m = 1$ under varying values of ε . As ε decreases, the numerical solution converges toward the expected result. Remarkably, even for a relatively high $\varepsilon = 0.01$, the approximation exhibits good accuracy, and further reductions in ε produce only marginal improvements. A comparison to the LDG formulation is also provided in Fig. 5 that clearly shows that the choice of $\varepsilon = 0.01$ is reasonable, since we know that the LDG formulation is relatively exact for $m = 1$.

For $m = 1$, the diffusion corresponds to the classical heat equation, which yields smooth solutions for all $t > 0$. As a result, no limiters were applied in this case. However, increasing m introduces nonlinear diffusion effects, which lead to the formation of steep gradients in the solution after a certain time $t > 0$. To address this, the two previously introduced limiters were employed. Specifically, if the first limiter (minmod) flags a troubled cell, the two neighboring cells are also limited by projecting their solutions to first-order polynomials. This conservative strategy enhances stability and helps reduce oscillations.

Despite this aggressive limiting, substantial oscillations still occur for larger nonlinearities such as $m = 6, 7, 8$. In contrast, for smaller values of m , the solution remains mostly free of spurious oscillations when starting from smooth initial data (compare Fig. 6).

Case 2 For relative non-smooth initial data, such as the Barenblatt solution at $t = 1$ s, the numerical scheme is able to capture the general diffusion behavior of the solution. However, as in the smooth case, increasing the nonlinearity significantly amplifies oscillations, particularly near the free boundary. As a result, the accurate resolution of the free boundary evolution cannot be reliably achieved. Nevertheless, in regions where the solution remains smooth, the method still delivers acceptable accuracy, especially for cases with $m \leq 3$ as can be seen in Fig. 7.

Noteworthy are the computation times for this relatively fine grid with $N_{\text{cells}} = 320$ which are significantly lower than those observed when using the LDG formulation. This is somewhat counterintuitive, given the introduction of the artificial parameter ε and its

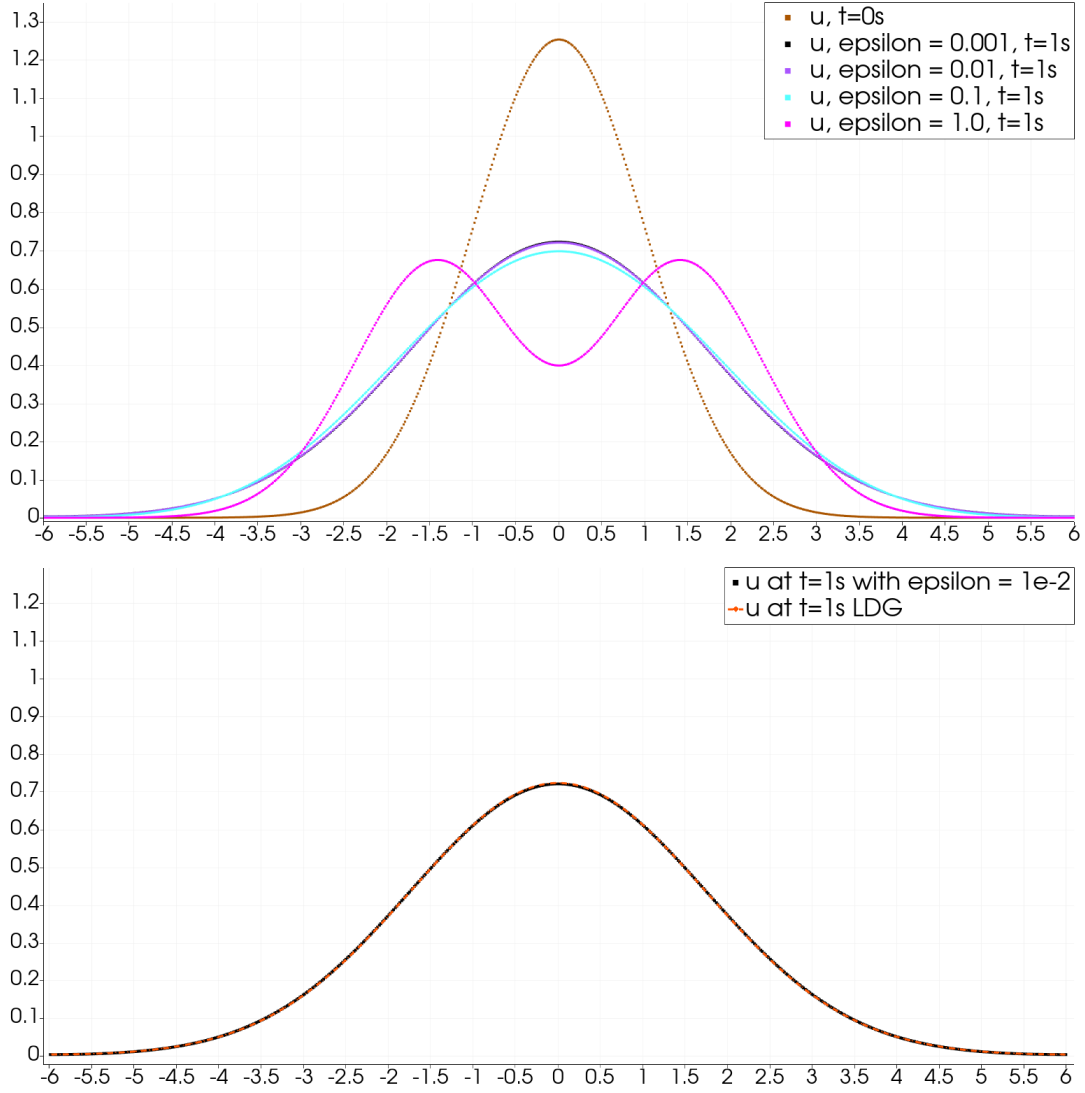


Fig. 5: Results for the hyperbolic system with varying ϵ and a comparison to the LDG method for the exponent $m = 1$

expected impact on time step restrictions. This performance advantage was observed consistently for $\epsilon = 0.01$ under single-core computations. Implying a use-case for which the nonlinearity is not as great the hyperbolic system can be of advantage for smooth initial data.

Case 3 Considering now the two-Box initial condition

$$u_0(x) = \begin{cases} 1, & \text{if } x \in (-3.7, -0.7) \cup (0.7, 3.7), \\ 0, & \text{otherwise,} \end{cases}$$

the hyperbolic system fails to produce acceptable solutions. The presence of huge discontinuities and therefore steep gradients leads to severe oscillations that cannot be sufficiently suppressed, even with the aggressive application of the previously introduced limiters. As a result, the numerical scheme is unable to capture the correct behavior of the solution for this type of discontinuous initial data, see Figure: 8.

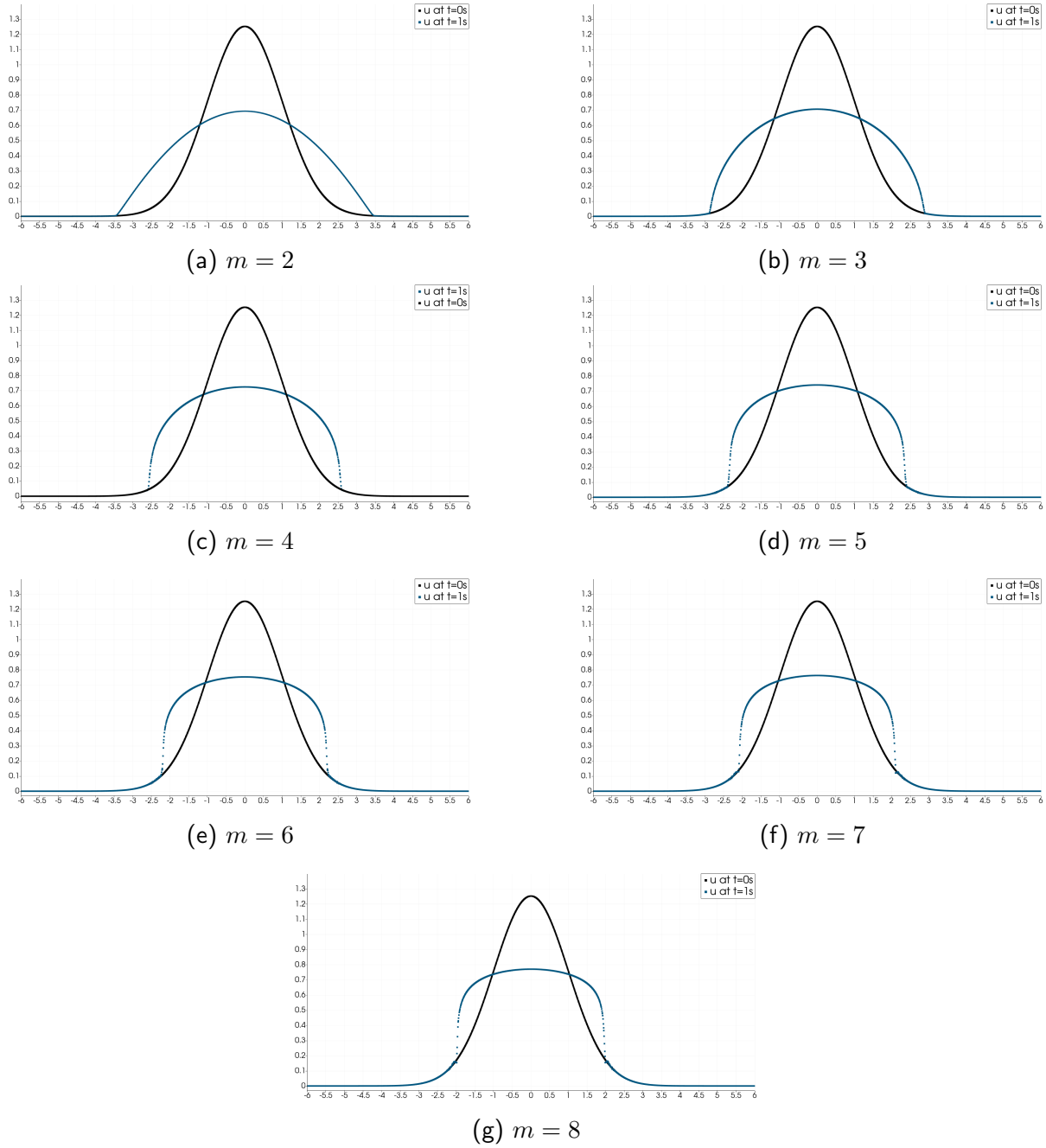


Fig. 6: Numerical results for varying nonlinearity factors with smooth initial data and polynomial degree of $l = 3$ calculated until $t = 1$ s

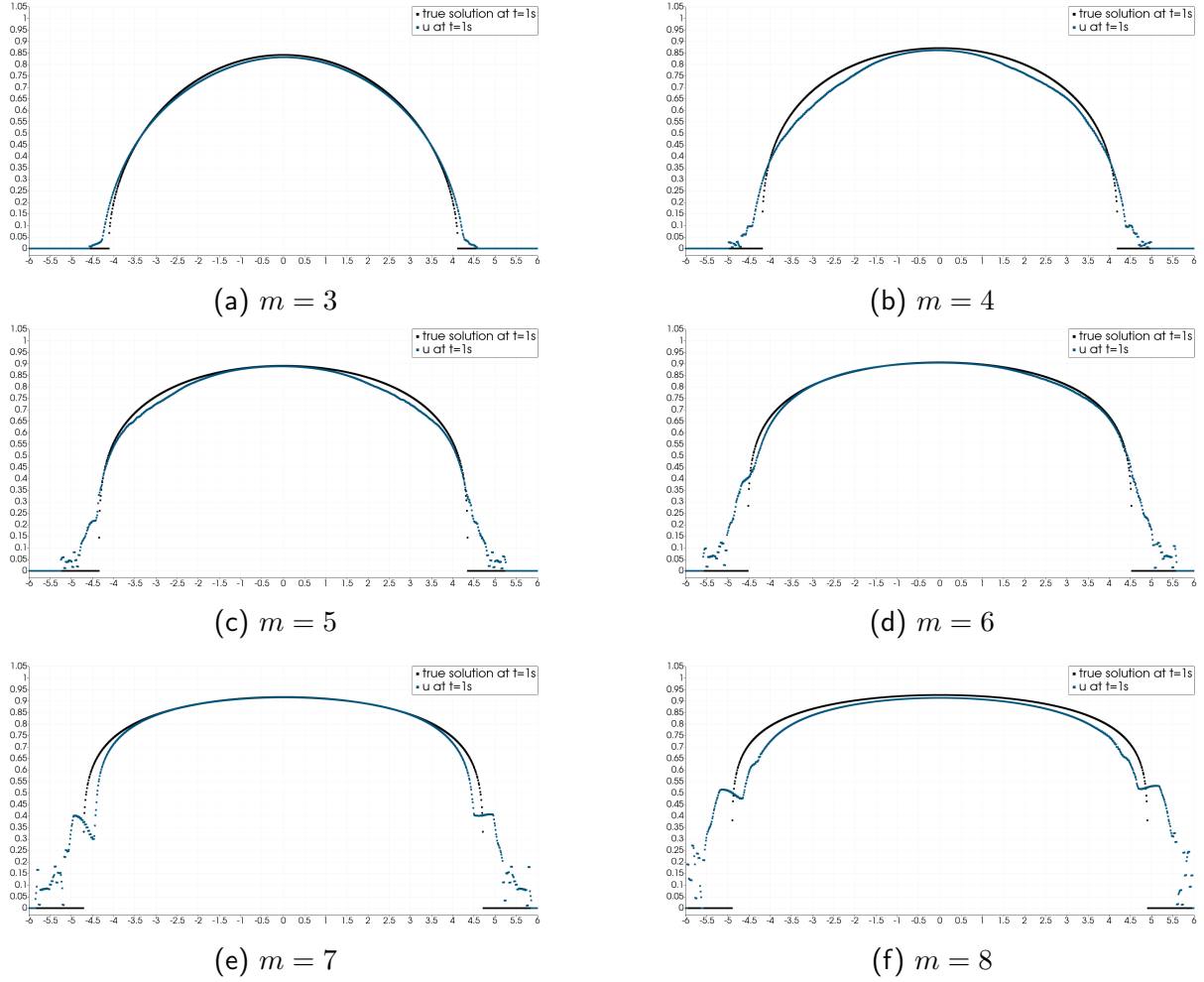


Fig. 7: Numerical results for varying nonlinearity factors with relative smooth initial data and polynomial degree of $l = 3$ calculated until $t = 2s$

Case 4 The final test case aims to investigate whether the hyperbolic system is capable of representing the waiting time phenomenon. For this purpose, the following specially designed initial condition is used

$$u_0(x) = \begin{cases} \cos x, & \text{if } x \in \left(-\frac{\pi}{2}, \frac{\pi}{2}\right), \\ 0, & \text{otherwise.} \end{cases}$$

At early times $t > 0s$, when the solution remains close to the initial data, the hyperbolic system successfully reproduces the waiting time phenomenon, even with a relatively small parameter value of $\varepsilon = 0.01$ (see Fig. 9). However, as time progresses, small oscillations begin to appear despite the strong enforcement of the modified limiters. These oscillations grow in magnitude over time but remain localized near the kinks in the solution. Despite these oscillations, the numerical solution appears to capture the qualitative behavior of the porous medium equation (PME) accurately and is capable of producing sharp transitions at the free boundary. This case is particularly noteworthy due to the high nonlinearity exponent $m = 8$. In comparison, the hyperbolic system was unable to

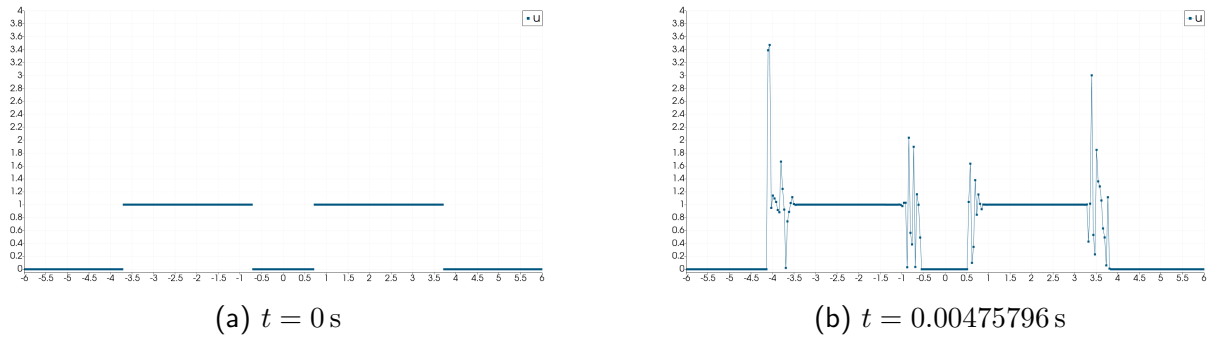


Fig. 8: Numerical results for varying nonlinearity factors with smooth initial data and polynomial degree of $l = 3$ calculated until $t = 1$ s

accurately capture the sharp free boundary for the Barenblatt initial data with the same exponent at $t = 1$ s (compare Fig. 7f), which supports the observation that the choice of initial data plays a crucial role in the accuracy and stability of the simulation.

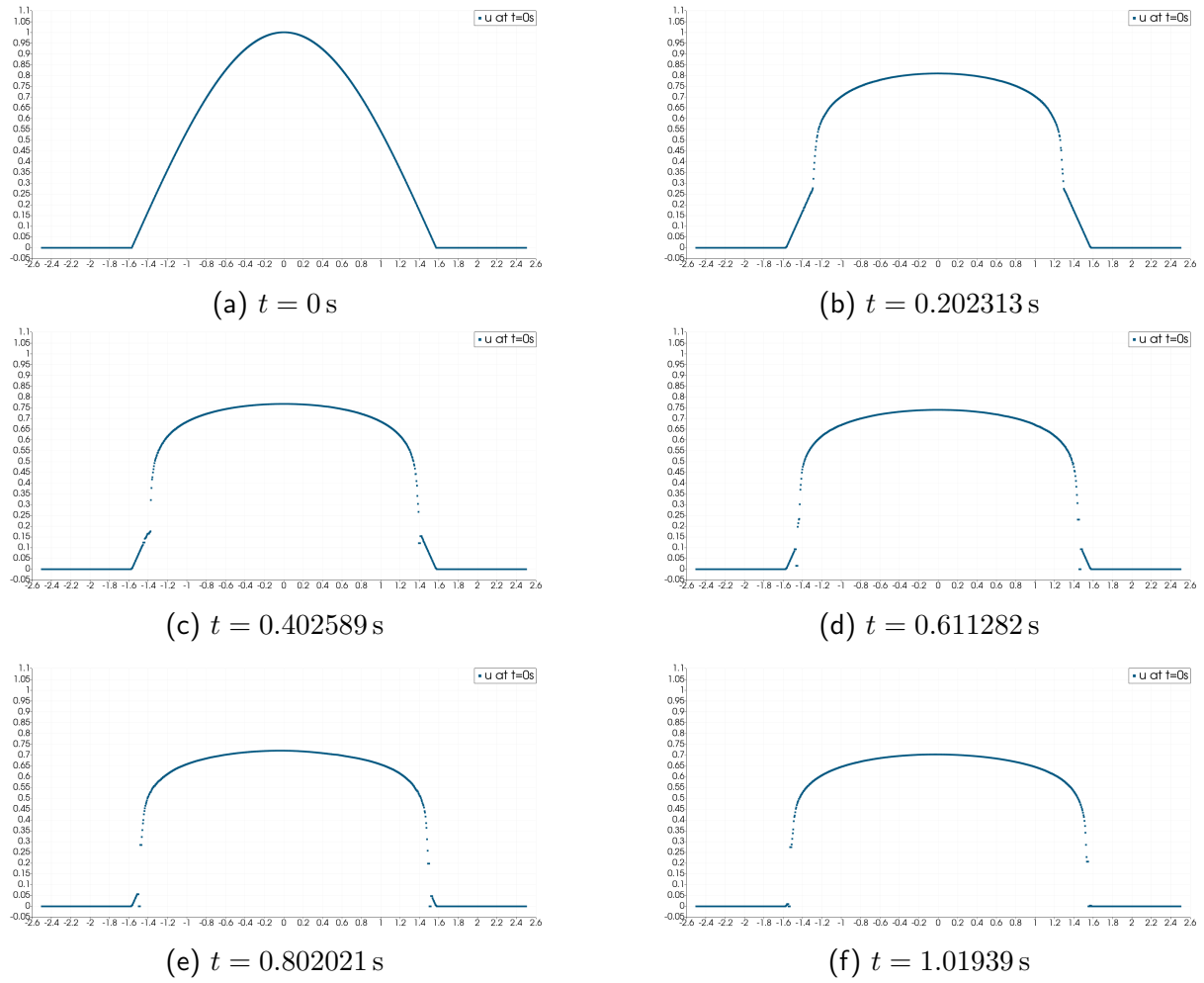


Fig. 9: The numerical results gained from the hyperbolic system displaying waiting time phenomenon for polynomial degree of $l = 3$ and nonlinearity exponent $m = 8$

5 Conclusion

In this work, we have numerically investigated the porous medium equation using two distinct discretization strategies: the Local Discontinuous Galerkin (LDG) method and a hyperbolic relaxation system. The implementation was carried out with a focus on positivity preservation, limiter design, and adaptive tolerance thresholds, ensuring both stability and physical accuracy in degenerate regimes.

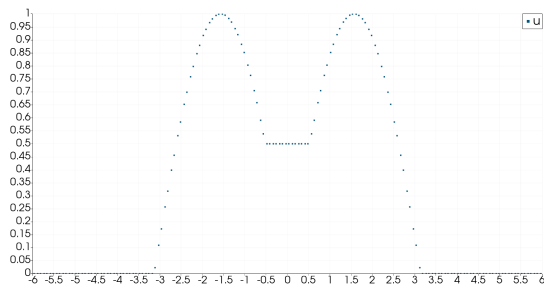
Our results demonstrate that the LDG method, when equipped with appropriately tuned limiters, can provide accurate and robust solutions for a wide range of nonlinearity exponents m , though challenges remain for high-order schemes under strong degeneracy. The hyperbolic formulation, on the other hand, exhibits greater robustness with respect to oscillations and is capable of capturing key phenomena such as waiting time behavior, even for small values of the relaxation parameter ε .

Through a systematic study of convergence behavior, limiter activity, and solution structure, we have shown that the choice of initial data and boundary treatment significantly influences the observed accuracy and stability. In particular, differences in experimental order of convergence compared to prior works such as Zhang and Shu can largely be attributed to the full-domain treatment used here.

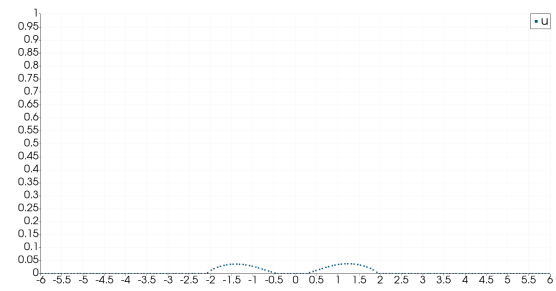
References

- [1] On some unsteady motions of a fluid and a gas in a porous medium. *Prikl. Mat. Makh.*, 16:67–78, 1952.
- [2] Bernardo Cockburn and Chi-Wang Shu. The local discontinuous galerkin method for time-dependent convection-diffusion systems. *SIAM Journal on Numerical Analysis*, 35(6):2440–2463, 1998.
- [3] Sigal Gottlieb, Chi-Wang Shu, and Eitan Tadmor. Strong stability-preserving high-order time discretization methods. *SIAM Review*, 43(1):89–112, 2001.
- [4] Shi Jin, Lorenzo Pareschi, and Giuseppe Toscani. Diffusive relaxation schemes for multiscale discrete-velocity kinetic equations. *SIAM Journal on Numerical Analysis*, 35:2405–2439, 12 1998.
- [5] David A. Kopriva. *Algorithms for Scientists and Engineers*. Springer Dordrecht, 1st edition, 2009.
- [6] Tatsuyuki Nakaki and Kenji Tomoeda. A finite difference scheme for some nonlinear diffusion equations in an absorbing medium: Support splitting phenomena. *SIAM J. Numerical Analysis*, 40:945–964, 09 2002.
- [7] Stanley Osher. Convergence of generalized muscl schemes. *SIAM Journal on Numerical Analysis*, 22(5):947–961, 1985.
- [8] Philip Rosenau and Shoshana Kamin. Thermal waves in an absorbing and convecting medium. *Physica D: Nonlinear Phenomena*, 8(1):273–283, 1983.
- [9] Qiang Zhang and Zi-Long Wu. Numerical simulation for porous medium equation by local discontinuous galerkin finite element method. *Journal of Scientific Computing*, 38(2):127–148, Feb 2009.

6 Appendix



(a) $t = 0 \text{ s}$



(b) $t = 0.710025 \text{ s}$

Fig. 10: Divorce of the support with parameters $m = 1.98$ and $p = 0.02$. Calculated for $N_{cells} = 80$ and a polynomial degree of $l = 2$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	4.52e-03	-	1.12e-02	-	227	336	56
20	3.05e-03	0.57	6.30e-03	0.83	1302	1332	222
40	3.21e-03	-0.08	4.91e-03	0.36	10092	12868	883
80	3.19e-03	0.01	3.95e-03	0.31	41119	42649	3530
160	3.18e-03	0.00	3.50e-03	0.18	167100	166384	14115
320	3.18e-03	0.00	3.28e-03	0.09	664708	1605483	56455

Error calculation results for $l = 1, m = 8$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	2.28e-04	-	6.45e-04	-	509	365	82
20	5.37e-05	2.09	9.30e-05	2.79	3375	404	328
40	5.23e-05	0.04	6.80e-05	0.45	8426	7851	1309
80	5.20e-05	0.01	5.75e-05	0.24	53905	23397	5234
160	5.19e-05	0.00	5.37e-05	0.10	233769	67510	20935
320	5.19e-05	0.00	5.29e-05	0.02	820520	393031	83738

Error calculation results for $l = 2, m = 8$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.40e-04	-	9.53e-04	-	1009	557	110
20	5.26e-07	8.05	1.86e-06	9.00	3955	2743	437
40	4.59e-07	0.20	6.22e-07	1.58	20872	19004	1745
80	4.59e-07	0.00	4.92e-07	0.34	66962	52520	6977
160	4.59e-07	0.00	4.70e-07	0.07	327605	265718	27908
320	4.59e-07	0.00	4.70e-07	0.00	1521531	339425	1025272

Error calculation results for $l = 3, m = 8$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	2.61e-05	-	1.67e-04	-	1085	807	137
20	3.12e-08	9.71	7.14e-08	11.19	6433	3276	546
40	3.06e-08	0.03	3.63e-08	0.98	23034	26129	2181
80	3.08e-08	-0.01	3.20e-08	0.18	1208	102158	95678
160	3.08e-08	0.00	3.14e-08	0.03	418511	282164	34885
320	3.08e-08	0.00	3.14e-08	0.00	1101816	1475461	139537

Error calculation results for $l = 4, m = 8$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.20e-05	-	7.81e-05	-	12578	9315	1636
20	4.21e-09	11.48	1.04e-08	12.88	67883	39246	6541
40	4.17e-09	0.01	4.63e-09	1.17	358207	290520	26164
80	4.13e-09	0.01	4.27e-09	0.12	1315959	983439	104653
160	4.13e-09	0.00	4.92e-09	-0.20	5415132	3868565	418608
320	4.07e-09	0.00	4.25e-09	0.00	24891778	19679230	1674432

Error calculation results for $l = 5, m = 8$

Tab. 5: Error calculation results for the endtime $t = 1.05$ s, $m = 8$ and various polynomial degrees l for the LDG system the Barenblatt function at $t = 1$ s as initial data

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.09e-02	-	2.11e-02	-	102	126	21
20	2.60e-03	2.07	5.46e-03	1.95	1155	1488	83
40	6.29e-04	2.05	1.44e-03	1.93	5731	5709	330
80	1.57e-04	2.01	3.32e-04	2.11	19171	16462	1318
160	3.91e-05	2.00	8.02e-05	2.05	82244	72648	5269
320	9.79e-06	2.00	1.97e-05	2.02	307524	267161	21074

$l = 1, m = 3$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	9.15e-04	-	5.64e-03	-	235	95	32
20	4.32e-05	4.40	2.56e-04	4.46	1129	740	124
40	5.18e-06	3.06	4.37e-05	2.55	4947	2961	494
80	6.41e-07	3.01	4.98e-06	3.13	13940	11853	1976
160	8.07e-08	2.99	5.96e-07	3.06	75289	47413	7903
320	1.01e-08	3.00	7.29e-08	3.03	461077	189655	31610

$l = 2, m = 3$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	5.98e-04	-	3.51e-03	-	456	288	42
20	2.82e-06	7.73	1.77e-05	7.63	2824	2479	165
40	1.51e-07	4.23	1.50e-06	3.57	11778	9795	659
80	9.34e-09	4.01	8.22e-08	4.19	39231	27082	2635
160	5.93e-10	3.98	4.83e-09	4.09	190875	117858	10537
320	5.26e-10	0.17	1.07e-08	-1.15	722761	455974	42147

$l = 3, m = 3$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.97e-04	-	1.61e-03	-	734	448	52
20	1.45e-07	10.41	9.72e-07	10.69	2408	2433	206
40	3.16e-09	5.52	4.61e-08	4.40	9671	9797	824
80	1.12e-10	4.81	1.28e-09	5.17	36854	42260	3293
160	7.59e-11	0.57	8.65e-10	0.56	153971	161134	13171
320	1.01e-10	-0.41	9.52e-10	-0.14	606102	639989	52683

$l = 4, m = 3$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	6.60e-05	-	6.38e-04	-	7044	3708	618
20	2.17e-08	11.57	1.44e-07	12.12	50055	43906	2470
40	1.81e-10	6.90	9.24e-10	7.28	195788	155101	9879
80	5.55e-11	1.71	5.58e-10	0.73	650785	463878	39513
160	1.47e-10	-1.41	1.53e-09	-1.46	3028715	1859780	158049
320	4.88e-11	1.60	2.54e-10	2.59	11331375	7311999	632194

$l = 5, m = 3$

Tab. 6: Error calculations for final time $t = 1.05$ s, $m = 3$ and varying polynomial degree for the LDG system the Barenblatt function at $t = 1$ s as initial data

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	8.84e-02	-	3.58e-01	-	2071	2111	352
20	4.84e-02	0.87	3.78e-01	-0.08	18794	19201	1387
40	3.63e-02	0.42	2.57e-01	0.56	87938	81809	5528
80	3.04e-02	0.25	1.71e-01	0.58	341070	300362	22097
160	2.14e-02	0.50	1.17e-01	0.55	1290441	1116830	88370
320	1.49e-02	0.52	8.23e-02	0.51	5122249	4424667	353464

$$l = 1, m = 3$$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.69e-01	-	6.67e-01	-	6081	561	514
20	2.25e-01	-0.41	1.25e+00	-0.91	31713	12386	2065
40	2.33e-01	-0.05	1.77e+00	-0.50	117348	49671	8279
80	2.57e-01	-0.14	3.11e+00	-0.82	498766	198603	33131
160	3.84e-01	-0.58	6.51e+00	-1.06	2137409	793967	132509
320	5.27e-01	-0.46	1.16e+01	-0.83	7894659	2829346	530013

$$l = 2, m = 3$$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	1.67e-01	-	6.04e-01	-	5920	4116	680
20	9.45e-02	0.82	4.40e-01	0.46	45536	33434	2755
40	4.70e-02	1.01	2.90e-01	0.60	194827	134580	11042
80	1.85e-02	1.35	1.41e-01	1.04	777279	492690	44180
160	1.58e-02	0.22	1.32e-01	0.10	3100555	1923621	176726
320	8.46e-03	0.90	8.88e-02	0.57	12215401	7572357	706916

$$l = 3, m = 3$$

N	L^2 Error	EOC_{L^2}	L^∞ Error	EOC_{L^∞}	#1st Lim.	#2nd Lim.	#steps
10	2.25e-01	-	9.43e-01	-	11703	7606	848
20	1.68e-01	0.42	1.04e+00	-0.14	42595	40745	3441
40	9.50e-02	0.82	8.21e-01	0.34	172107	175815	13798
80	4.87e-02	0.97	3.68e-01	1.16	681005	708505	55222
160	4.27e-02	0.19	3.04e-01	0.27	2780841	2829610	220890
320	3.04e-02	0.49	2.26e-01	0.43	10707423	10660451	883592

$$l = 4, m = 3$$

Tab. 7: Error calculations for varying polynomial degrees and the nonlinearity factor $m = 3$ at $t = 2$ s, for the LDG system the Barenblatt function at $t = 1$ s as initial data